STAT 417 Final Code Work

Group 8

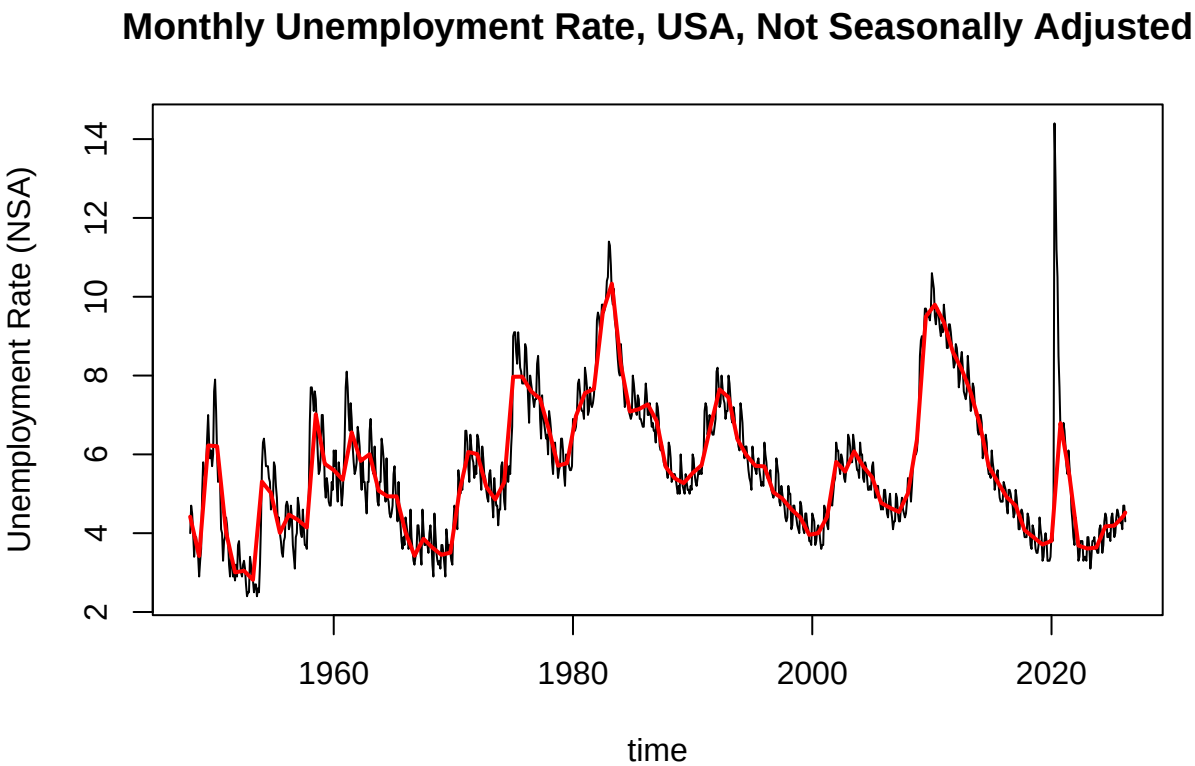
2026-04-27

Initial data pull/clean

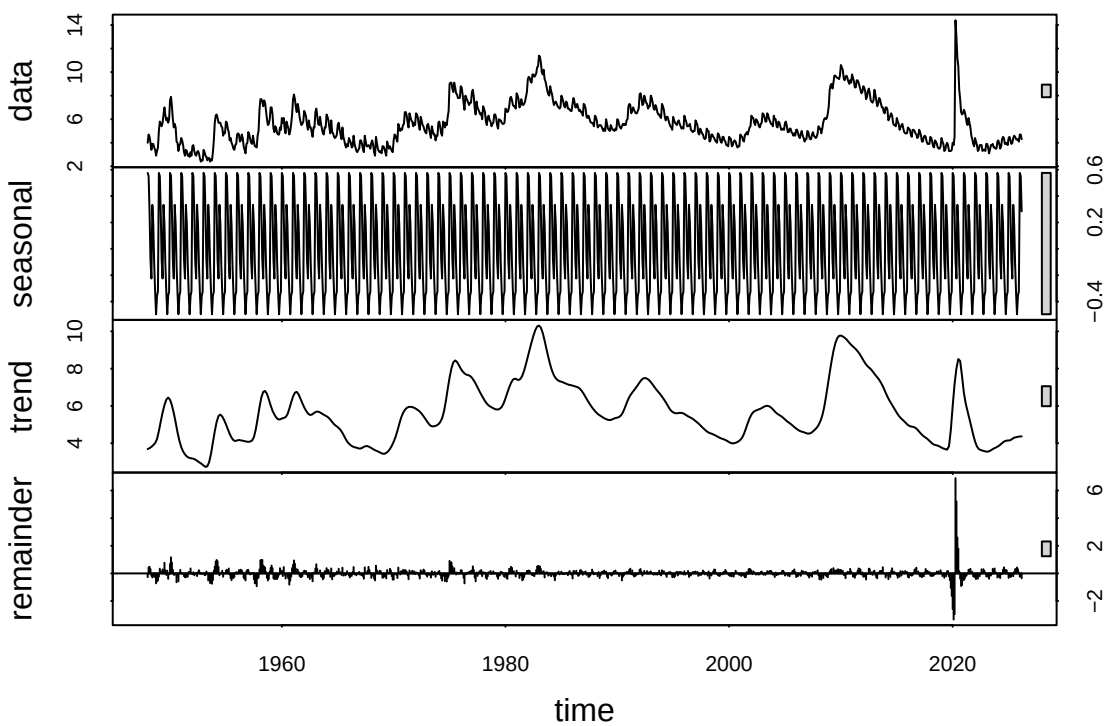
```
#read the data from the fed website
library(readr)
url <- "https://fred.stlouisfed.org/graph/fredgraph.csv?id=UNRATENSA"
unrate_data <- read_csv(url)

#convert to time series
unrate_ts <- ts(unrate_data$UNRATENSA, start = c(1948, 1), frequency = 12)
#interpolate the NA values
library(forecast)
unrate_ts_interp <- na.interp(unrate_ts)

#Seasonally adjusted data (for comparison maybe)
urlb <- "https://fred.stlouisfed.org/graph/fredgraph.csv?id=UNRATE"
unrate_data_sa <- read_csv(urlb)

#convert to time series
unratesa_ts <- ts(unrate_data_sa$UNRATE, start = c(1948, 1), frequency = 12)
#interpolate the NA values
library(forecast)
unratesa_ts_interp <- na.interp(unratesa_ts)
```

ts analysis

```
#full data set and smoothing
plot(unrate_ts_interp, ylab="Unemployment Rate (NSA)", xlab = "time", col = "black", main = "Monthly Un
lines(lowess(unrate_ts_interp, f=0.01), lwd=2, col="red")
```

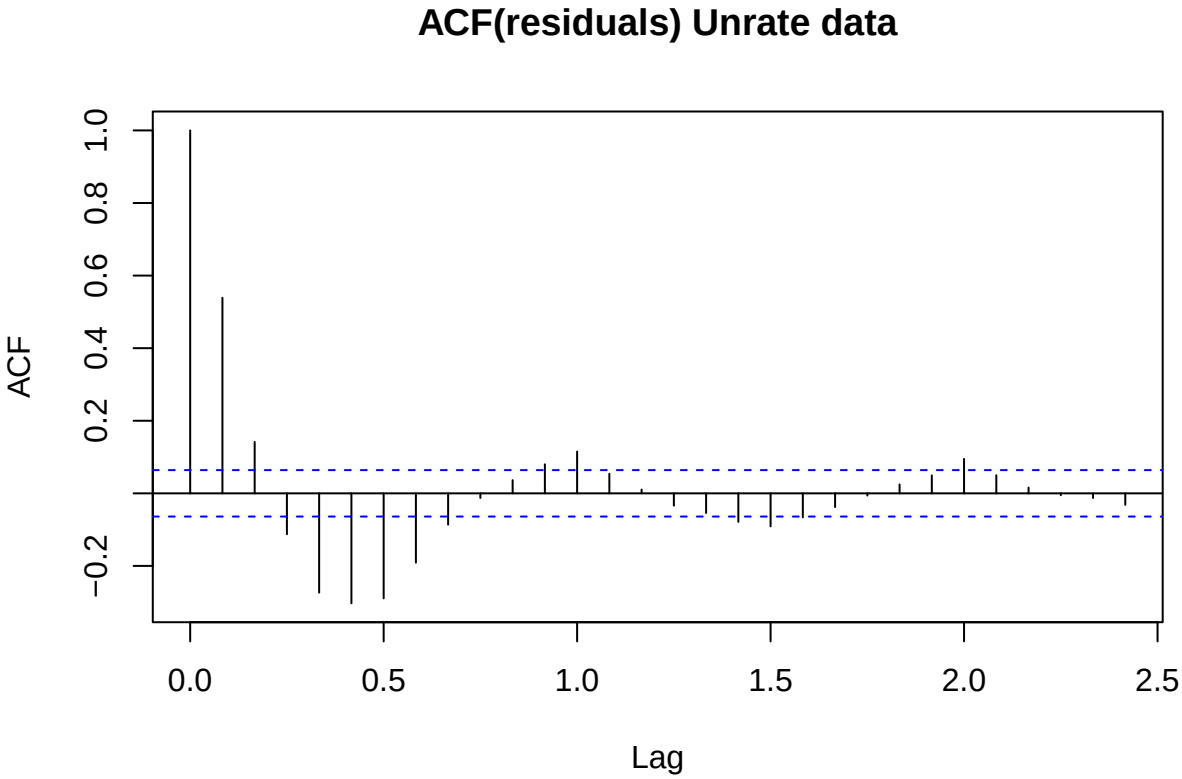



```r
#stl analysis
unrate_stl <- stl(unrate_ts_interp, s.window = "periodic")
plot(unrate_stl)
```




```r
season <- unrate_stl$time.series[,1]
trend <- unrate_stl$time.series[,2]
res <- unrate_stl$time.series[,3]
```

```r
acf(res, main = "ACF(residuals) Unrate data")
```



We can see seasonality in the tiny spikes, probably should blow this graph up on the ppt.

ACF reveals time dependence of the residuals, so we need to model the residuals.

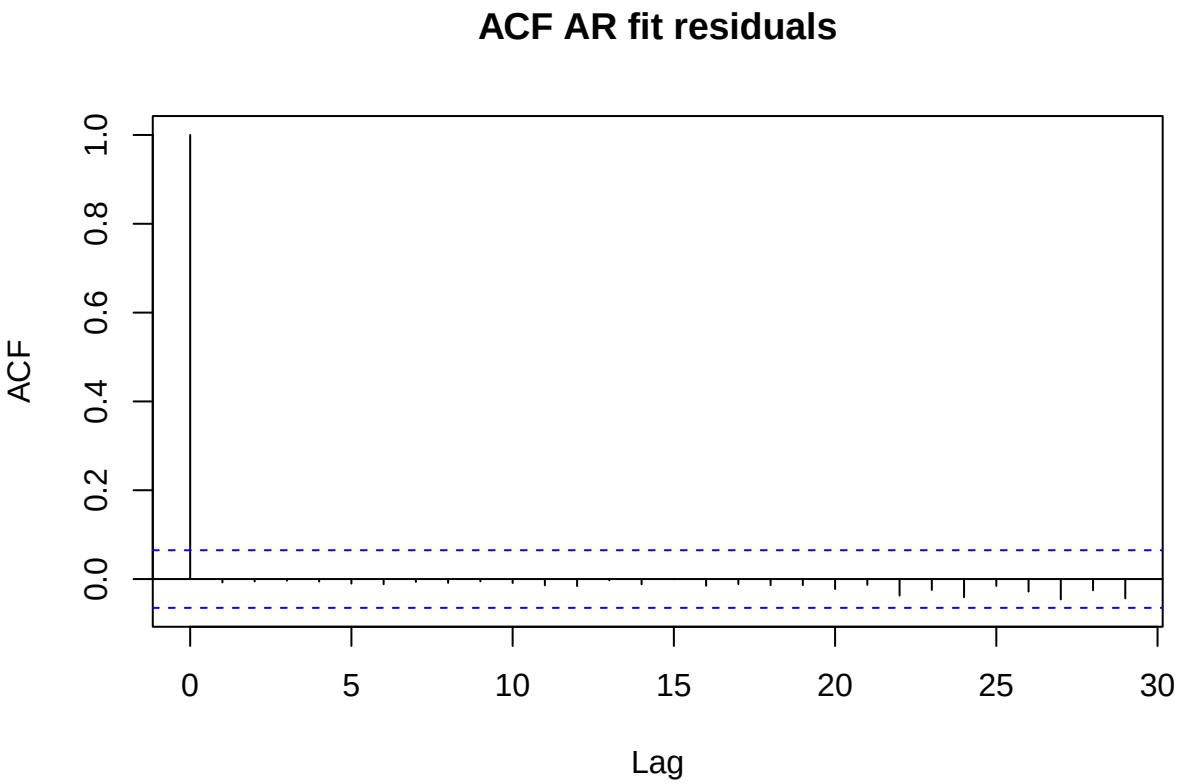
Residual AR fit model

```r
#get the order of the AR fit
fit_ar <- ar.yw(res, order.max = NULL)
fit_ar
```

```
##
## Call:
## ar.yw.default(x = res, order.max = NULL)
##
## Coefficients:
##       1        2        3        4        5        6        7        8
##  0.5390  -0.1878  -0.0777  -0.1762  -0.0508  -0.1600  -0.0535  -0.0702
##       9       10       11       12       13       14       15       16
## -0.0747  -0.0875  -0.0626   0.0171  -0.1280  -0.0235  -0.0851  -0.0188
##      17       18       19       20       21       22       23       24
## -0.0776  -0.0515  -0.0488  -0.0375  -0.0470  -0.0208  -0.0613   0.0722
##      25
## -0.0894
##
## Order selected 25  sigma^2 estimated as  0.1151
```

```r
#look at the residuals for this new model
fit_ar <- ar.yw(res, order.max = NULL)
```

```r
fit_resid <- ts(fit_ar$resid)
p<- fit_ar$order
acf(fit_resid[-c(1:p)], main = "ACF AR fit residuals") #remove the first p(order) observations, the res
```



Trend model

```r
time <- 1:length(trend)
dat.trend <- data.frame(trend = trend, time = time)

# trig fit
fit_trig <- lm(trend ~
                 cos(2*pi*time/12) + sin(2*pi*time/12) +
                 cos(4*pi*time/12) + sin(4*pi*time/12) +
                 cos(6*pi*time/12) + sin(6*pi*time/12),
               data = dat.trend)
summary(fit_trig)
```

```
##
## Call:
## lm(formula = trend ~ cos(2 * pi * time/12) + sin(2 * pi * time/12) +
##     cos(4 * pi * time/12) + sin(4 * pi * time/12) + cos(6 * pi *
##     time/12) + sin(6 * pi * time/12), data = dat.trend)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -2.9156 -1.2961 -0.1835  0.9681  4.6620
##
## Coefficients:
```

```
##                          Estimate Std. Error t value Pr(>|t|)
## (Intercept)             5.6601717  0.0523184 108.187   <2e-16 ***
## cos(2 * pi * time/12) -0.0046277  0.0740284  -0.063    0.950
## sin(2 * pi * time/12) -0.0093150  0.0739502  -0.126    0.900
## cos(4 * pi * time/12)  0.0034942  0.0739891   0.047    0.962
## sin(4 * pi * time/12) -0.0058515  0.0739893  -0.079    0.937
## cos(6 * pi * time/12)  0.0034437  0.0740282   0.047    0.963
## sin(6 * pi * time/12) -0.0007344  0.0739496  -0.010    0.992
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.603 on 932 degrees of freedom
## Multiple R-squared:  3.294e-05,  Adjusted R-squared:  -0.006405
## F-statistic: 0.005117 on 6 and 932 DF,  p-value: 1
```

```r
# polynomial fits
#I() forces literal interpretation of the eqn
fit_poly10 <- lm(trend ~ time + I(time^2) + I(time^3) + I(time^4) + I(time^5) + I(time^6) + I(time^7) +

fit_poly9 <- lm(trend ~ time + I(time^2) + I(time^3) + I(time^4) + I(time^5) + I(time^6) + I(time^7) + I

fit_poly8 <- lm(trend ~ time + I(time^2) + I(time^3) + I(time^4) + I(time^5) + I(time^6) + I(time^7) + I

summary(fit_poly10)
```

```
##
## Call:
## lm(formula = trend ~ time + I(time^2) + I(time^3) + I(time^4) +
##     I(time^5) + I(time^6) + I(time^7) + I(time^8) + I(time^9) +
##     I(time^10), data = dat.trend)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -2.1209 -0.8791 -0.1588  0.6790  4.0306
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  5.484e+00  4.151e-01  13.212  < 2e-16 ***
## time        -8.840e-02  3.036e-02  -2.912  0.00368 **
## I(time^2)    1.500e-03  7.276e-04   2.061  0.03956 *
## I(time^3)   -6.109e-06  8.115e-06  -0.753  0.45177
## I(time^4)   -3.030e-08  4.986e-08  -0.608  0.54359
## I(time^5)    3.462e-10  1.840e-10   1.881  0.06025 .
## I(time^6)   -1.272e-12  4.249e-13  -2.993  0.00283 **
## I(time^7)    2.420e-15  6.180e-16   3.916 9.65e-05 ***
## I(time^8)   -2.559e-18  5.499e-19  -4.653 3.75e-06 ***
## I(time^9)    1.426e-21  2.732e-22   5.221 2.20e-07 ***
## I(time^10)  -3.276e-25  5.806e-26  -5.643 2.22e-08 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.12 on 928 degrees of freedom
## Multiple R-squared:  0.5144, Adjusted R-squared:  0.5092
## F-statistic: 98.31 on 10 and 928 DF,  p-value: < 2.2e-16
```

```r
summary(fit_poly9)
```

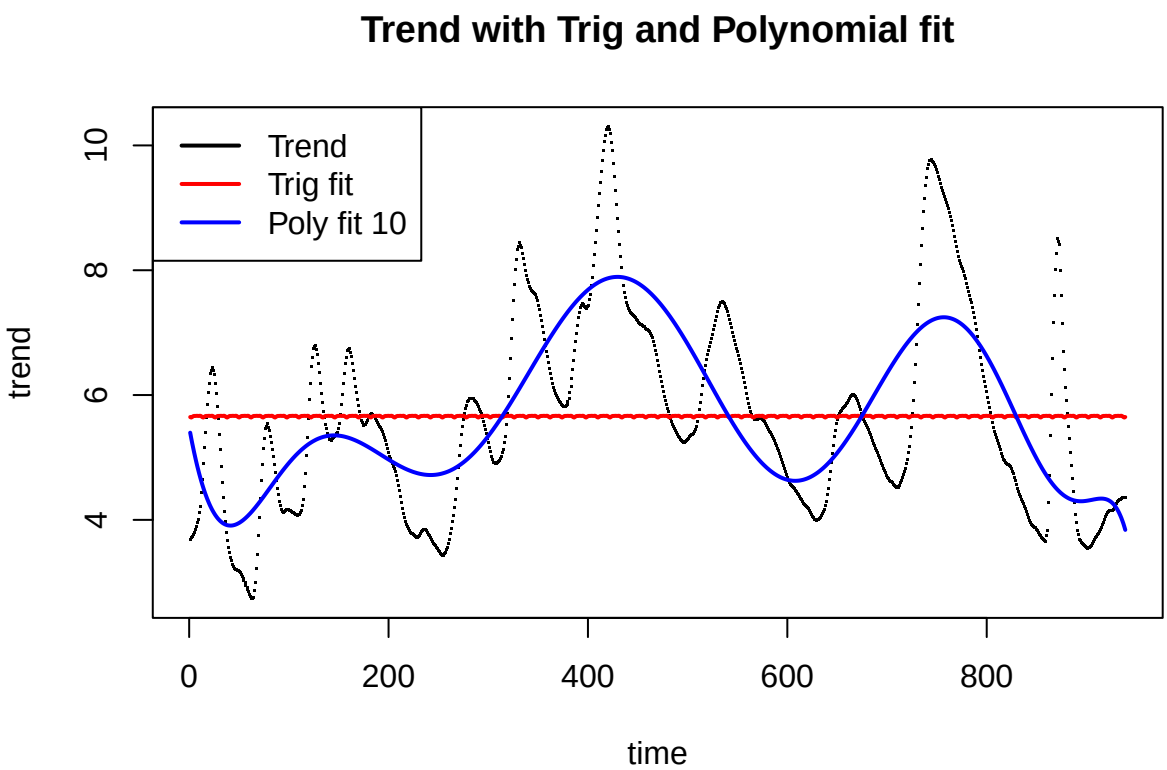
```
##
## Call:
## lm(formula = trend ~ time + I(time^2) + I(time^3) + I(time^4) +
##     I(time^5) + I(time^6) + I(time^7) + I(time^8) + I(time^9),
##     data = dat.trend)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -2.5979 -0.8635 -0.1588  0.6841  3.9585
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  6.486e+00  3.814e-01  17.006  < 2e-16 ***
## time        -2.023e-01  2.305e-02  -8.779  < 2e-16 ***
## I(time^2)    4.742e-03  4.537e-04  10.453  < 2e-16 ***
## I(time^3)   -4.580e-05  4.114e-06 -11.133  < 2e-16 ***
## I(time^4)    2.277e-07  2.023e-08  11.254  < 2e-16 ***
## I(time^5)   -6.404e-10  5.835e-11 -10.975  < 2e-16 ***
## I(time^6)    1.059e-12  1.015e-13  10.434  < 2e-16 ***
## I(time^7)   -1.018e-15  1.046e-16  -9.739  < 2e-16 ***
## I(time^8)    5.270e-19  5.876e-20   8.969  < 2e-16 ***
## I(time^9)   -1.134e-22  1.387e-23  -8.176 9.59e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.138 on 929 degrees of freedom
## Multiple R-squared:  0.4978, Adjusted R-squared:  0.4929
## F-statistic: 102.3 on 9 and 929 DF,  p-value: < 2.2e-16
```

```r
summary(fit_poly8)
```

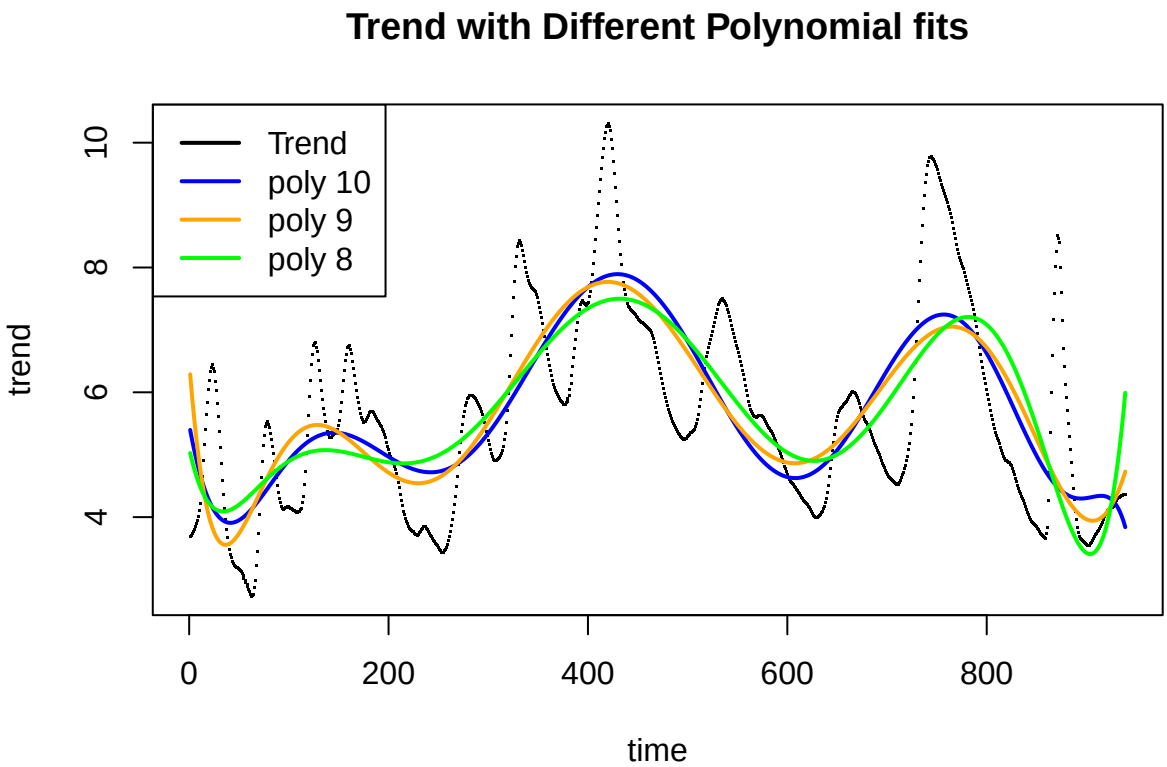
```
##
## Call:
## lm(formula = trend ~ time + I(time^2) + I(time^3) + I(time^4) +
##     I(time^5) + I(time^6) + I(time^7) + I(time^8), data = dat.trend)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -1.7865 -0.9008 -0.0317  0.6954  4.1089
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  5.097e+00  3.534e-01  14.425  < 2e-16 ***
## time        -7.246e-02  1.728e-02  -4.194 3.01e-05 ***
## I(time^2)    1.725e-03  2.731e-04   6.317 4.12e-10 ***
## I(time^3)   -1.594e-05  1.960e-06  -8.132 1.34e-15 ***
## I(time^4)    7.314e-08  7.463e-09   9.800  < 2e-16 ***
## I(time^5)   -1.805e-10  1.608e-11 -11.231  < 2e-16 ***
## I(time^6)    2.437e-13  1.966e-14  12.399  < 2e-16 ***
## I(time^7)   -1.694e-16  1.271e-17 -13.323  < 2e-16 ***
## I(time^8)    4.735e-20  3.374e-21  14.033  < 2e-16 ***
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.178 on 930 degrees of freedom
## Multiple R-squared:  0.4616, Adjusted R-squared:  0.457
## F-statistic: 99.68 on 8 and 930 DF,  p-value: < 2.2e-16
```

```r
# compare trig vs polynomial
#fitted() applies the coefficients to the model for each data point
plot(time, trend, pch=".", main="Trend with Trig and Polynomial fit")
lines(time, fitted(fit_trig), col="red", lwd=2)
lines(time, fitted(fit_poly10), col="blue", lwd=2)
legend("topleft",
       legend=c("Trend", "Trig fit","Poly fit 10"),
       col=c("black", "red", "blue"),
       lwd=2)
```




```r
# compare polynomial degrees
plot(time, trend, pch=".", main="Trend with Different Polynomial fits")
lines(time, fitted(fit_poly10), col="blue", lwd=2)
lines(time, fitted(fit_poly9), col="orange", lwd=2)
lines(time, fitted(fit_poly8), col="green", lwd=2)
legend("topleft",
       legend=c("Trend", "poly 10","poly 9","poly 8"),
       col=c("black", "blue", "orange","green"),
       lwd=2)
```



Trig fit is not great, the plot is not even close and the p-values are high. Polynomial fit is good: of the three, degree 10 has the lowest residual standard error.

Seasonal model

```
time <- 1:length(season)
dat.season <- data.frame(season = season, time = time)

fit_season1 <- lm(season ~
                    cos(2*pi*time/12) + sin(2*pi*time/12),
                  data = dat.season)

summary(fit_season1)
```

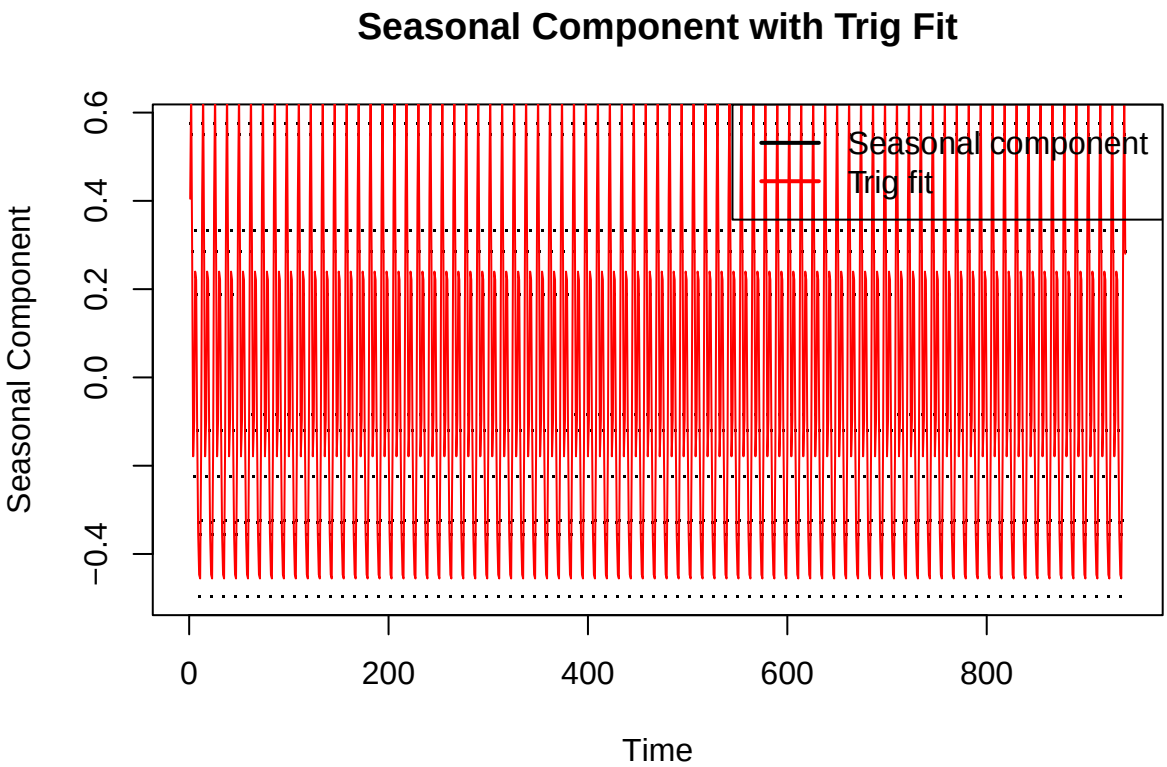
```
##
## Call:
## lm(formula = season ~ cos(2 * pi * time/12) + sin(2 * pi * time/12),
##     data = dat.season)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.41997 -0.20964 -0.01918  0.29451  0.46633
##
## Coefficients:
##                         Estimate Std. Error t value Pr(>|t|)
## (Intercept)            0.0008124  0.0092394   0.088 0.929949
## cos(2 * pi * time/12) -0.0495083  0.0130733  -3.787 0.000162 ***
## sin(2 * pi * time/12)  0.3026840  0.0130595  23.177  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##
## Residual standard error: 0.2831 on 936 degrees of freedom
## Multiple R-squared:  0.3706, Adjusted R-squared:  0.3693
## F-statistic: 275.6 on 2 and 936 DF,  p-value: < 2.2e-16
```

```r
fit_season2 <- lm(season ~
                  cos(2*pi*time/12) + sin(2*pi*time/12) +
                  cos(4*pi*time/12) + sin(4*pi*time/12) +
                  cos(6*pi*time/12) + sin(6*pi*time/12),
                data = dat.season)

summary(fit_season2)
```

```
##
## Call:
## lm(formula = season ~ cos(2 * pi * time/12) + sin(2 * pi * time/12) +
##     cos(4 * pi * time/12) + sin(4 * pi * time/12) + cos(6 * pi *
##     time/12) + sin(6 * pi * time/12), data = dat.season)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.157399 -0.081057  0.004208  0.094241  0.170761
##
## Coefficients:
##                         Estimate Std. Error t value Pr(>|t|)
## (Intercept)            6.604e-05  3.211e-03   0.021 0.983597
## cos(2 * pi * time/12) -5.051e-02  4.544e-03 -11.117  < 2e-16 ***
## sin(2 * pi * time/12)  3.016e-01  4.539e-03  66.453  < 2e-16 ***
## cos(4 * pi * time/12)  3.671e-02  4.541e-03   8.084 1.94e-15 ***
## sin(4 * pi * time/12)  3.400e-01  4.541e-03  74.875  < 2e-16 ***
## cos(6 * pi * time/12) -1.525e-01  4.544e-03 -33.559  < 2e-16 ***
## sin(6 * pi * time/12) -1.513e-02  4.539e-03  -3.333 0.000894 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.0984 on 932 degrees of freedom
## Multiple R-squared:  0.9243, Adjusted R-squared:  0.9238
## F-statistic:  1897 on 6 and 932 DF,  p-value: < 2.2e-16
```

```r
plot(time, season, pch=".", main="Seasonal Component with Trig Fit",
     xlab="Time", ylab="Seasonal Component")
lines(time, fitted(fit_season2), col="red")
legend("topright", legend=c("Seasonal component", "Trig fit"),
       col=c("black", "red"), lwd=2)
```

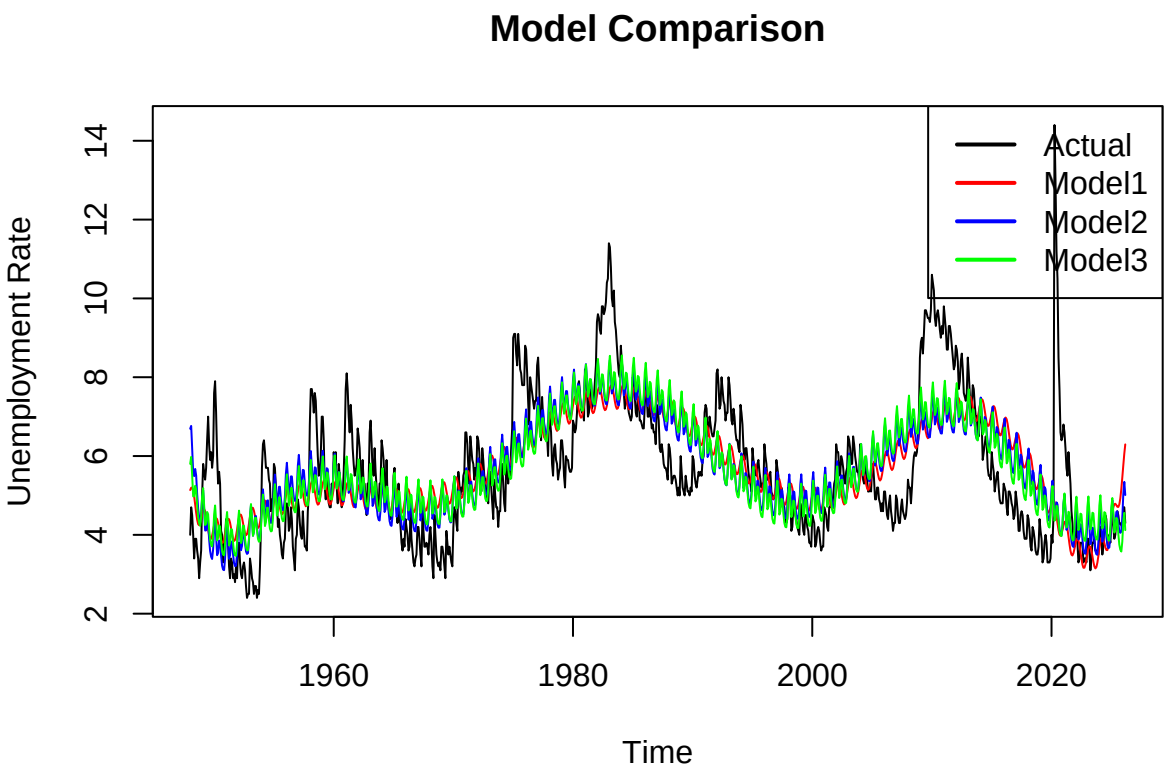


Graph is super ugly and compressed, but the more complex trig has higher R squared.

Combined Models

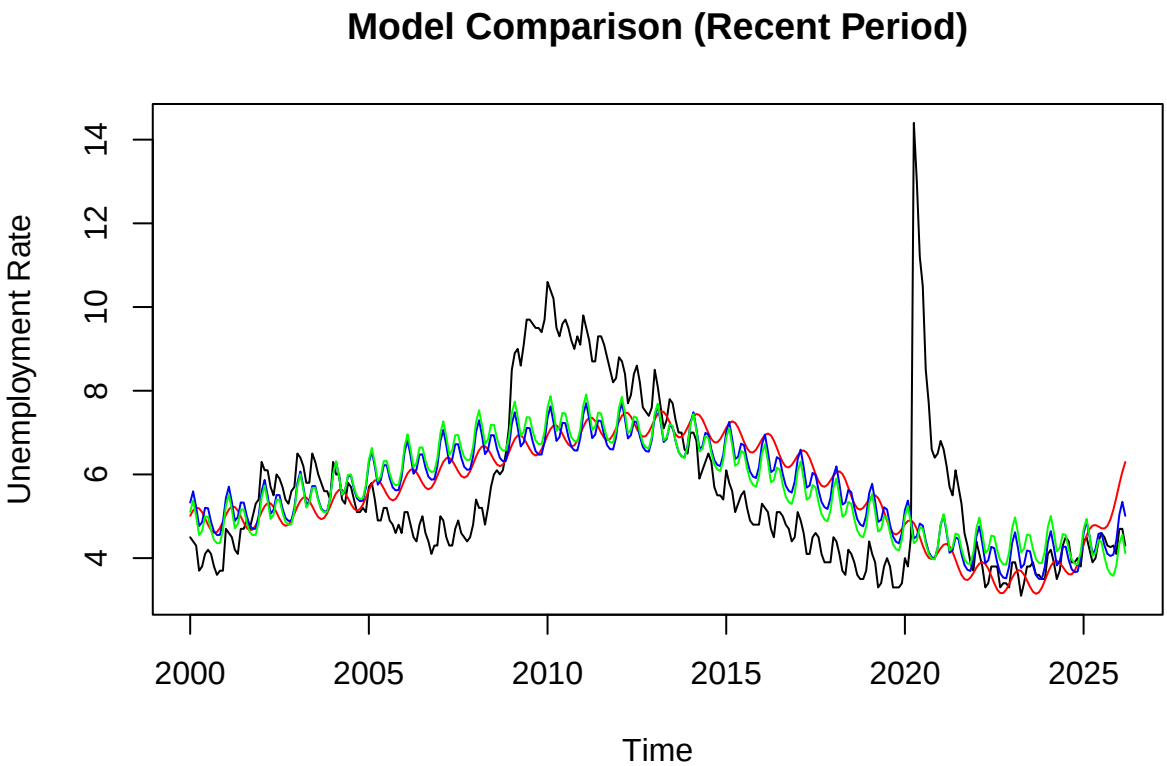
```
#model 1: poly 8 and simple trig
tr1 <- fitted(fit_poly8)

coef_s1 <- coef(fit_season1)

ss1 <- coef_s1[1] +
       coef_s1[2]*cos(2*pi*time/12) +
       coef_s1[3]*sin(2*pi*time/12)

yhat1 <- tr1 + ss1

#model 2: poly 9 and complex trig
tr2 <- fitted(fit_poly9)

coef_s2 <- coef(fit_season2)

ss2 <- coef_s2[1] +
       coef_s2[2]*cos(2*pi*time/12) +
       coef_s2[3]*sin(2*pi*time/12) +
       coef_s2[4]*cos(4*pi*time/12) +
       coef_s2[5]*sin(4*pi*time/12) +
       coef_s2[6]*cos(6*pi*time/12) +
       coef_s2[7]*sin(6*pi*time/12)

yhat2 <- tr2 + ss2
```

```r
#model 3: ploy 10 and complex trig
tr3 <- fitted(fit_poly10)

yhat3 <- tr3 + ss2

#plot the models
plot(unrate_ts_interp, col="black",
     main="Model Comparison",
     ylab="Unemployment Rate")

lines(ts(yhat1, start=start(unrate_ts_interp), frequency=12),
      col="red")

lines(ts(yhat2, start=start(unrate_ts_interp), frequency=12),
      col="blue")

lines(ts(yhat3, start=start(unrate_ts_interp), frequency=12),
      col="green")

legend("topright",
       legend=c("Actual", "Model1", "Model2", "Model3"),
       col=c("black", "red", "blue", "green"),
       lwd=2)
```




```r
#only recent years
start_plot <- c(2000,1)

plot(window(unrate_ts_interp, start=start_plot), col="black",
     main="Model Comparison (Recent Period)",
     ylab="Unemployment Rate")
```

```r
lines(window(ts(yhat1, start=start(unrate_ts_interp), frequency=12), start=start_plot),
      col="red")

lines(window(ts(yhat2, start=start(unrate_ts_interp), frequency=12), start=start_plot),
      col="blue")

lines(window(ts(yhat3, start=start(unrate_ts_interp), frequency=12), start=start_plot),
      col="green")
```



II. Time Series Data Analysis (Continued): Smoothing Methods & Forecasting

Naïve Method

The naïve method serves as a simple baseline forecast: each future value is set equal to the most recent observed value. For a non-seasonal series, the forecast for all future horizons h is:

$$\hat{X}_{t+h} = X_t$$

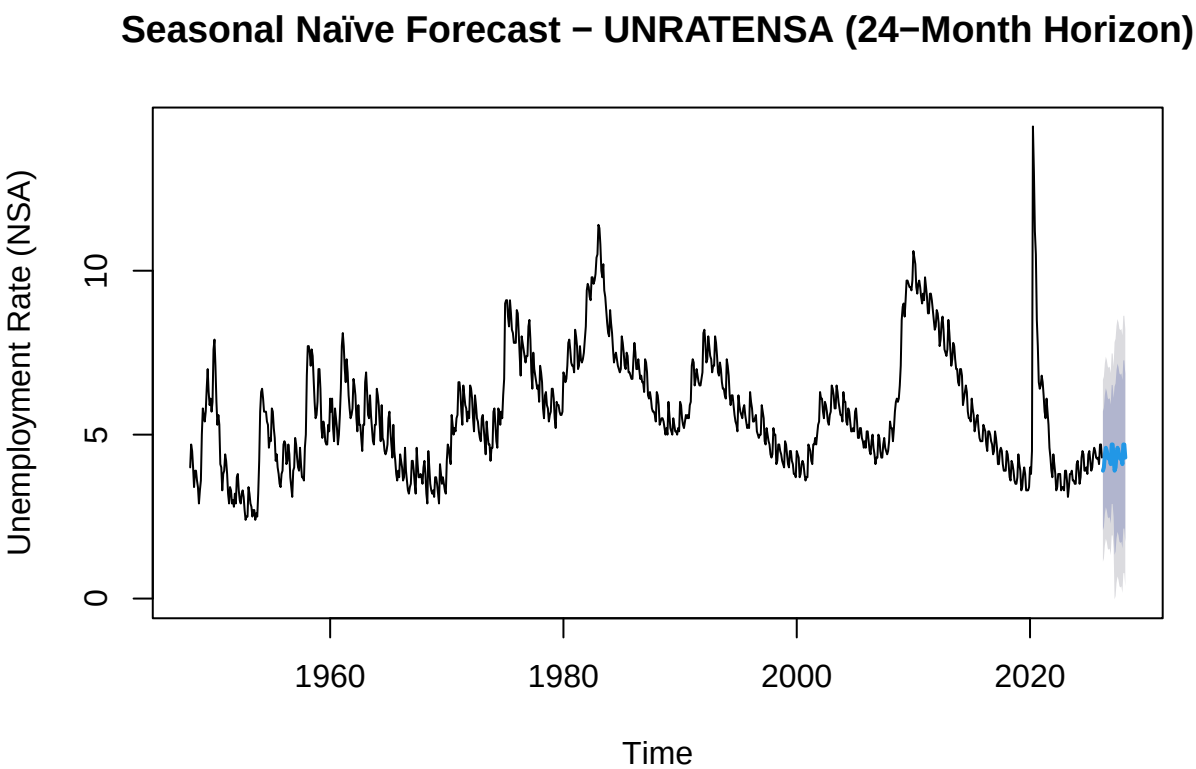
For a seasonal series (seasonal naïve), the forecast repeats the value from the same season in the previous cycle:

$$\hat{X}_{t+h} = X_{t+h-m}$$

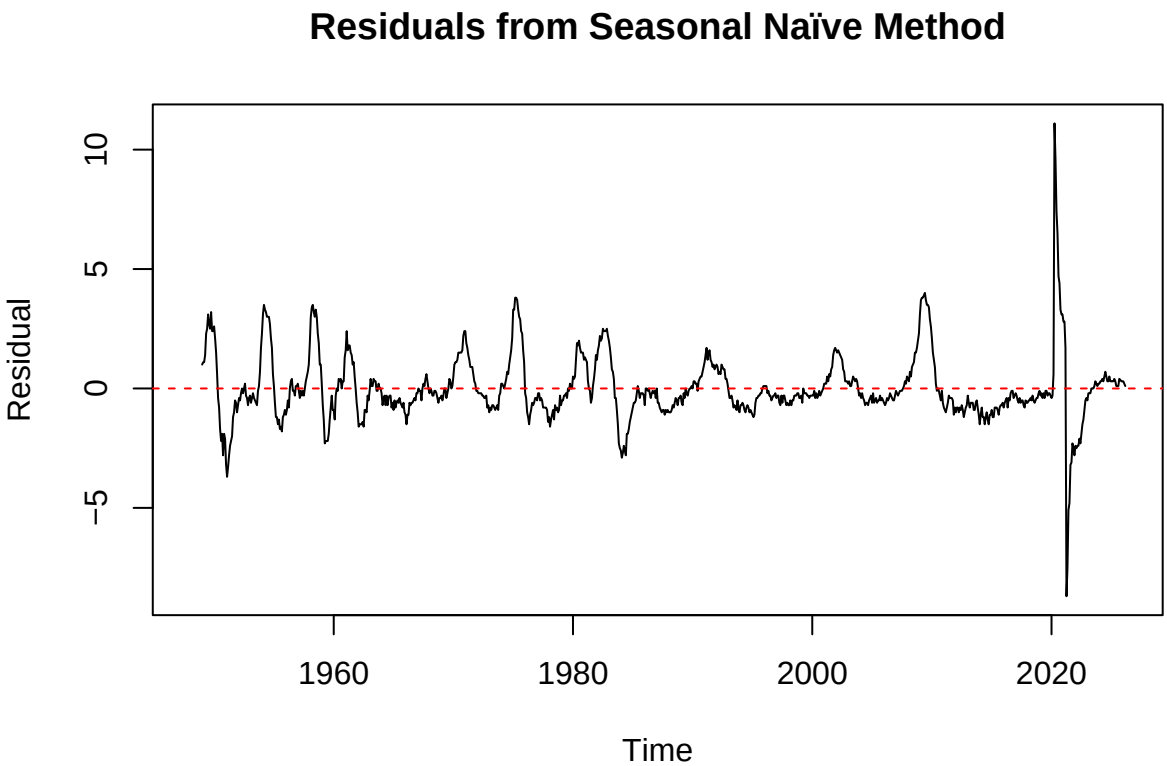
where m is the seasonal period. Because `UNRATENSA` exhibits strong monthly seasonality, the seasonal naïve method is the appropriate baseline.

```r
library(forecast)

# Seasonal naive forecast (12-month seasonality)
```

```r
naive_fit <- snaive(unrate_ts_interp, h = 24)

plot(naive_fit,
     main = "Seasonal Naïve Forecast - UNRATENSA (24-Month Horizon)",
     ylab = "Unemployment Rate (NSA)",
     xlab = "Time",
     col  = "black")
```




```r
# Residual diagnostics for naive baseline
naive_resid <- residuals(naive_fit)
plot(naive_resid,
     main = "Residuals from Seasonal Naïve Method",
     ylab = "Residual",
     xlab = "Time")
abline(h = 0, col = "red", lty = 2)
```



The seasonal naïve forecast simply projects the pattern of the previous year forward, providing a useful benchmark against which more sophisticated methods can be compared.

Exponential Smoothing

Exponential smoothing methods assign geometrically decreasing weights to past observations, giving the most recent data the greatest influence on forecasts. Three variants are applied here, chosen based on the characteristics of `UNRATENSA`.

Simple Exponential Smoothing (SES) SES is appropriate when a series has no consistent trend or seasonality — useful here as a reference. The smoothed estimate at time t is:

$$e_t = \alpha X_t + (1-\alpha)\, e_{t-1}, \quad 0 < \alpha < 1$$

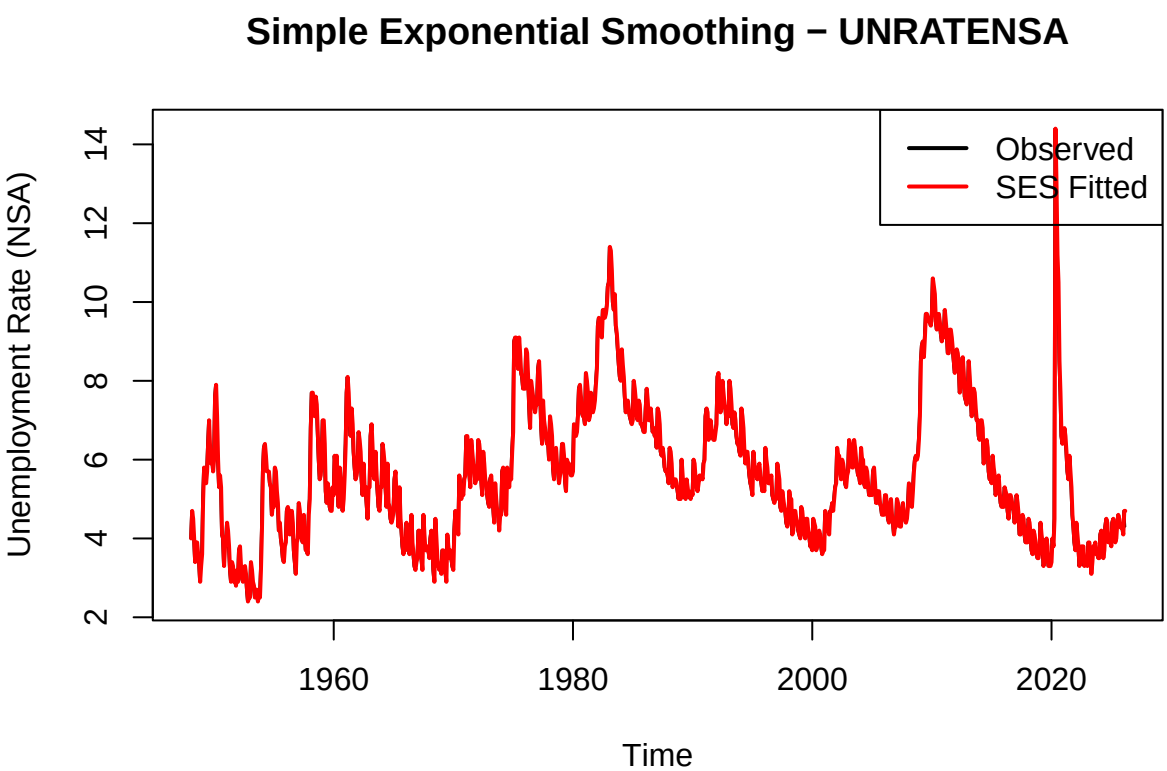
A small α weights historical observations more heavily; a large α reacts quickly to recent changes.

```r
# Simple exponential smoothing (no trend, no seasonality -- reference model)
ses_fit <- HoltWinters(unrate_ts_interp, beta = FALSE, gamma = FALSE)
ses_fit  # print smoothing parameters
```

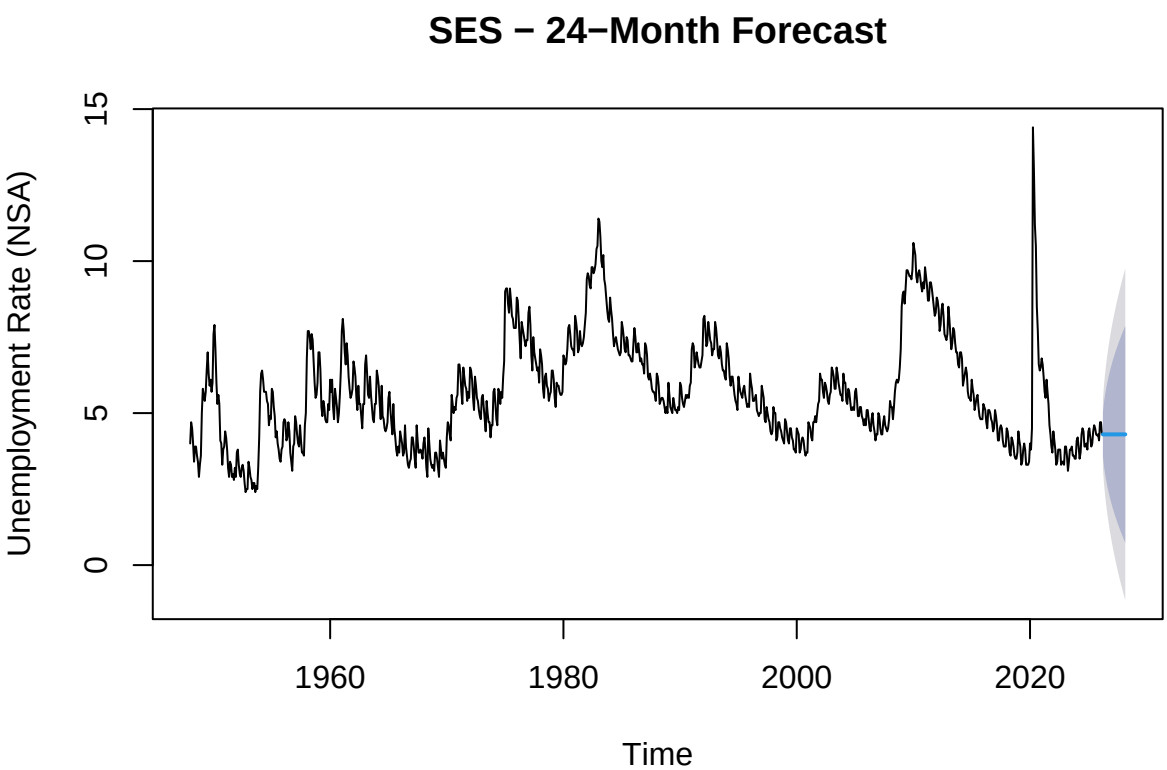
```
## Holt-Winters exponential smoothing without trend and without seasonal component.
##
## Call:
## HoltWinters(x = unrate_ts_interp, beta = FALSE, gamma = FALSE)
##
## Smoothing parameters:
##  alpha: 0.9999232
##  beta : FALSE
##  gamma: FALSE
```

```
##
## Coefficients:
##       [,1]
## a 4.300031
```

```r
plot(unrate_ts_interp,
     main = "Simple Exponential Smoothing - UNRATENSA",
     ylab = "Unemployment Rate (NSA)",
     xlab = "Time",
     col  = "black")
lines(ses_fit$fitted[, 1], col = "red", lwd = 2)
legend("topright",
       legend = c("Observed", "SES Fitted"),
       col    = c("black", "red"),
       lwd    = 2)
```




```r
# 24-month forecast
ses_forecast <- forecast(ses_fit, h = 24)
plot(ses_forecast,
     main = "SES - 24-Month Forecast",
     ylab = "Unemployment Rate (NSA)",
     xlab = "Time")
```



Holt's Exponential Smoothing (Trend, No Seasonality) Holt's method extends SES by also estimating the slope (trend) of the series using a second smoothing parameter β:

$$\ell_t = \alpha X_t + (1-\alpha)(\ell_{t-1} + b_{t-1})$$
$$b_t = \beta(\ell_t - \ell_{t-1}) + (1-\beta)\, b_{t-1}$$

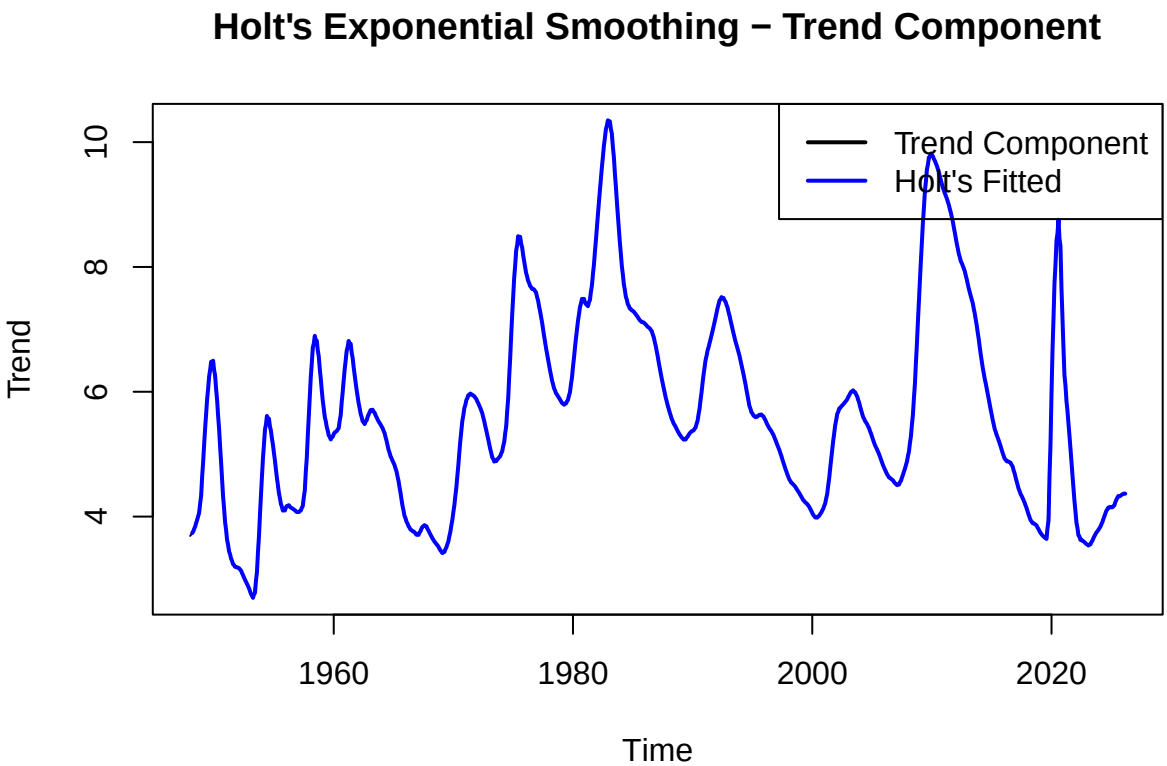
This is applied to the trend component extracted from the STL decomposition.

```
# Holt's method on the trend component
holts_fit <- HoltWinters(trend, gamma = FALSE)
holts_fit  # print alpha and beta
```

```
## Holt-Winters exponential smoothing with trend and without seasonal component.
##
## Call:
## HoltWinters(x = trend, gamma = FALSE)
##
## Smoothing parameters:
##  alpha: 0.9184004
##  beta : 1
##  gamma: FALSE
##
## Coefficients:
##           [,1]
## a 4.3655048485
## b 0.0009737063
```

```
plot(trend,
     main = "Holt's Exponential Smoothing - Trend Component",
```

```r
     ylab = "Trend",
     xlab = "Time",
     col  = "black")
lines(holts_fit$fitted[, 1], col = "blue", lwd = 2)
legend("topright",
       legend = c("Trend Component", "Holt's Fitted"),
       col    = c("black", "blue"),
       lwd    = 2)
```



Holt-Winters Exponential Smoothing (Trend + Seasonality) Because `UNRATENSA` has both trend and strong monthly seasonality, Holt-Winters (HW) is the most appropriate exponential smoothing method. HW adds a third smoothing parameter γ for the seasonal component:

$$\ell_t = \alpha(X_t - S_{t-m}) + (1-\alpha)(\ell_{t-1} + b_{t-1})$$
$$b_t = \beta(\ell_t - \ell_{t-1}) + (1-\beta)\, b_{t-1}$$
$$S_t = \gamma(X_t - \ell_t) + (1-\gamma)\, S_{t-m}$$

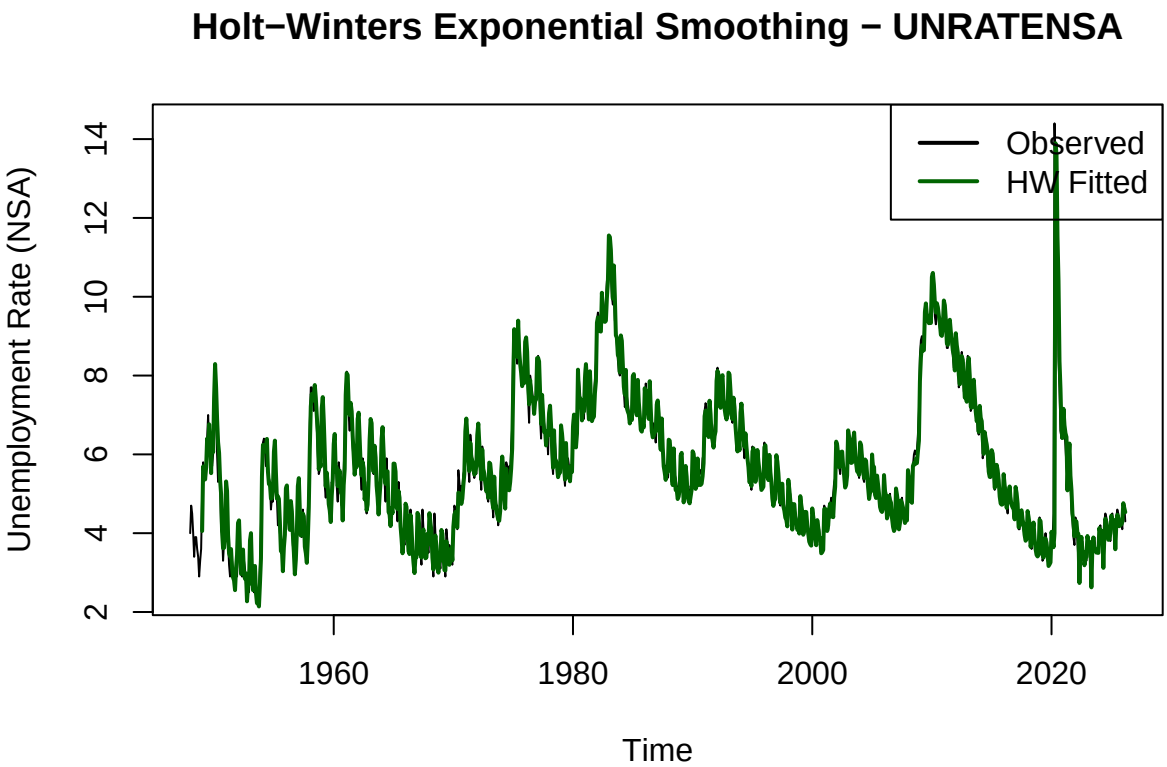
where $m = 12$ for monthly data.

```r
# Holt-Winters on the full UNRATENSA series
hw_fit <- HoltWinters(unrate_ts_interp)
hw_fit  # print alpha, beta, gamma
```

```
## Holt-Winters exponential smoothing with trend and additive seasonal component.
##
## Call:
## HoltWinters(x = unrate_ts_interp)
##
## Smoothing parameters:
```

```
##  alpha: 0.9325619
##  beta : 0.01093134
##  gamma: 1
##
## Coefficients:
##             [,1]
## a    4.329301618
## b   -0.008964252
## s1   0.070343008
## s2  -0.194581134
## s3   0.063102535
## s4   0.150125618
## s5  -0.045341574
## s6  -0.353325427
## s7  -0.412697628
## s8  -0.443505478
## s9  -0.469169327
## s10  0.194788226
## s11  0.147037263
## s12 -0.029301618
```

```r
# Plot fitted vs observed
plot(unrate_ts_interp,
     main = "Holt-Winters Exponential Smoothing - UNRATENSA",
     ylab = "Unemployment Rate (NSA)",
     xlab = "Time",
     col  = "black")
lines(hw_fit$fitted[, 1], col = "darkgreen", lwd = 2)
legend("topright",
       legend = c("Observed", "HW Fitted"),
       col    = c("black", "darkgreen"),
       lwd    = 2)
```



Forecast Results

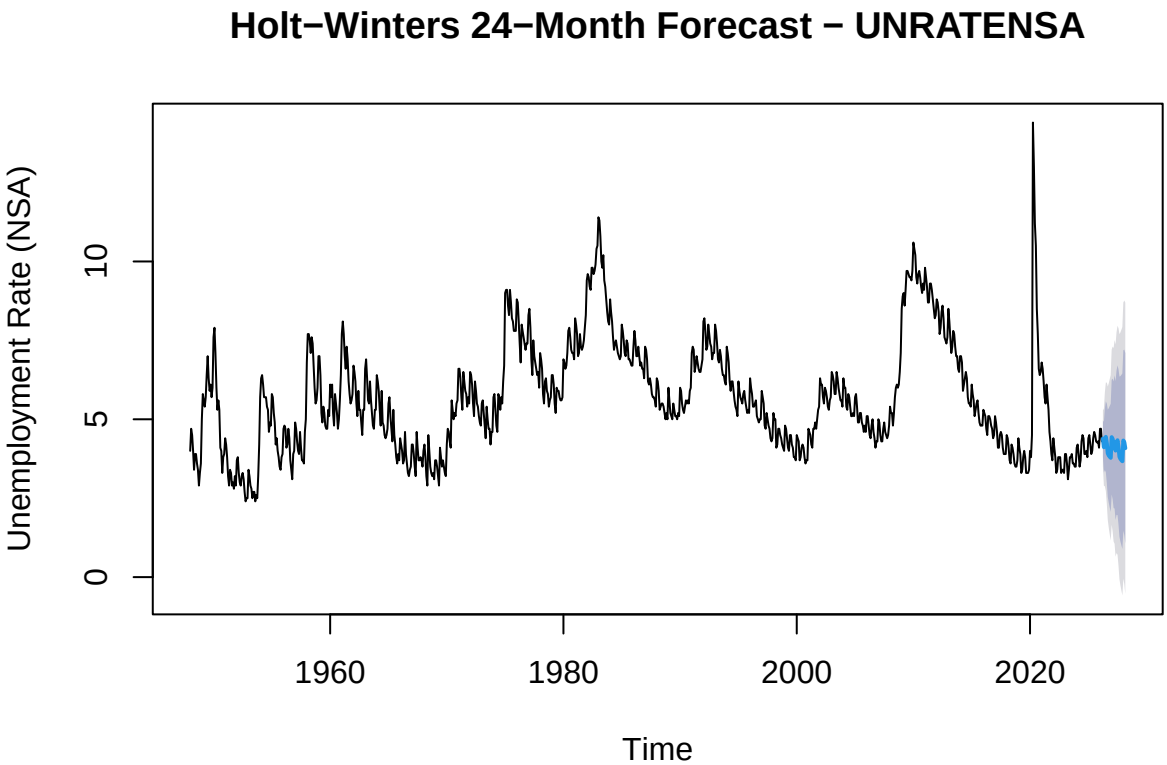
A 24-month ahead forecast is generated from the Holt-Winters model, representing the primary smoothing-based forecast for `UNRATENSA`.

```
hw_forecast <- forecast(hw_fit, h = 24)
hw_forecast  # print forecast table
```

```
##          Point Forecast     Lo 80    Hi 80       Lo 95    Hi 95
## Apr 2026       4.390680 3.8087204 4.972640  3.50064935 5.280711
## May 2026       4.116792 3.3169856 4.916598  2.89359368 5.339990
## Jun 2026       4.365511 3.3922447 5.338778  2.87702847 5.853994
## Jul 2026       4.443570 3.3204402 5.566700  2.72589101 6.161249
## Aug 2026       4.239139 2.9812516 5.497026  2.31536641 6.162911
## Sep 2026       3.922191 2.5402011 5.304180  1.80862005 6.035761
## Oct 2026       3.853854 2.3557432 5.351965  1.56269108 6.145017
## Nov 2026       3.814082 2.2060786 5.422086  1.35485298 6.273311
## Dec 2026       3.779454 2.0665672 5.492341  1.15981975 6.399088
## Jan 2027       4.434447 2.6207978 6.248097  1.66070968 7.208185
## Feb 2027       4.377732 2.4667699 6.288694  1.45516767 7.300297
## Mar 2027       4.192429 2.1870845 6.197773  1.12551921 7.259339
## Apr 2027       4.283109 2.1740781 6.392141  1.05762436 7.508594
## May 2027       4.009221 1.8109918 6.207450  0.64731963 7.371122
## Jun 2027       4.257940 1.9723771 6.543504  0.76247304 7.753408
## Jul 2027       4.335999 1.9647448 6.707254  0.70947856 7.962520
## Aug 2027       4.131568 1.6760788 6.587057  0.37622148 7.886914
## Sep 2027       3.814620 1.2761939 6.353045 -0.06756754 7.696807
## Oct 2027       3.746283 1.1260817 6.366485 -0.26096922 7.753536
## Nov 2027       3.706511 1.0055764 6.407446 -0.42421208 7.837234
```

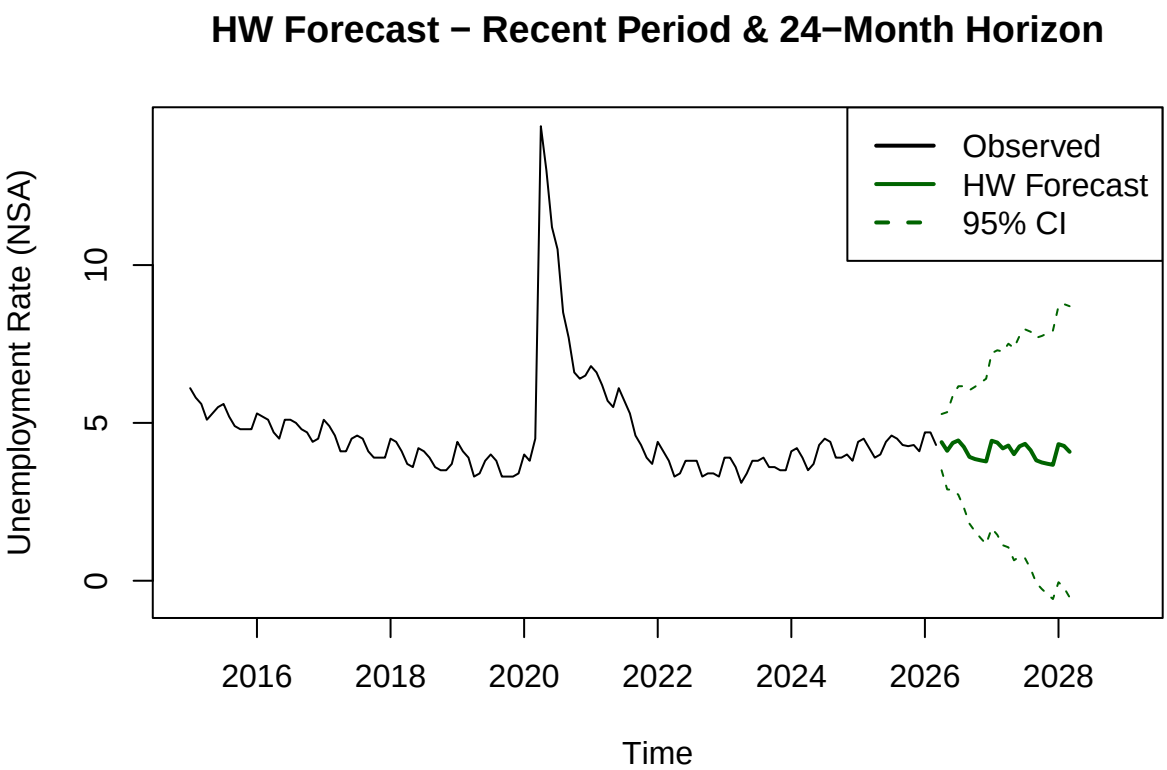
```
## Dec 2027       3.671883 0.8911542 6.452612 -0.58087469 7.924641
## Jan 2028       4.326876 1.4672016 7.186551 -0.04661872 8.700371
## Feb 2028       4.270161 1.3323083 7.208014 -0.22289698 8.763219
## Mar 2028       4.084858 1.0695236 7.100192 -0.52669805 8.696414
```

```r
# Full forecast plot
plot(hw_forecast,
     main = "Holt-Winters 24-Month Forecast - UNRATENSA",
     ylab = "Unemployment Rate (NSA)",
     xlab = "Time",
     col  = "black")
```




```r
# Zoomed view (recent period + forecast)
recent_start <- c(2015, 1)
plot(window(unrate_ts_interp, start = recent_start),
     xlim = c(2015, end(hw_forecast$mean)[1] + 1),
     ylim = range(c(
       window(unrate_ts_interp, start = recent_start),
       hw_forecast$lower, hw_forecast$upper
     ), na.rm = TRUE),
     main = "HW Forecast - Recent Period & 24-Month Horizon",
     ylab = "Unemployment Rate (NSA)",
     xlab = "Time",
     col  = "black")
lines(hw_forecast$mean,   col = "darkgreen", lwd = 2)
lines(hw_forecast$lower[, 2], col = "darkgreen", lty = 2)
lines(hw_forecast$upper[, 2], col = "darkgreen", lty = 2)
legend("topright",
       legend = c("Observed", "HW Forecast", "95% CI"),
       col    = c("black", "darkgreen", "darkgreen"),
       lty    = c(1, 1, 2),
```

```
        lwd     = 2)
```



Forecast Accuracy & Residual Diagnostics

```
# Accuracy metrics for each method
cat("=== Seasonal Naive ===\n")
```

```
## === Seasonal Naive ===
```

```
print(accuracy(naive_fit))
```

```
##                       ME     RMSE      MAE       MPE    MAPE MASE      ACF1
## Training set 0.007403186 1.415716 0.931675 -2.442651 15.9323    1 0.9118357
```

```
cat("\n=== Simple Exponential Smoothing ===\n")
```

```
##
## === Simple Exponential Smoothing ===
```

```
ses_fc_train <- forecast(ses_fit, h = 1)
print(accuracy(ses_forecast))
```

```
##                        ME      RMSE       MAE        MPE     MAPE      MASE
## Training set 0.0003198867 0.5679757 0.3586128 -0.4186922 6.494335 0.3849119
##                     ACF1
## Training set 0.05680498
```

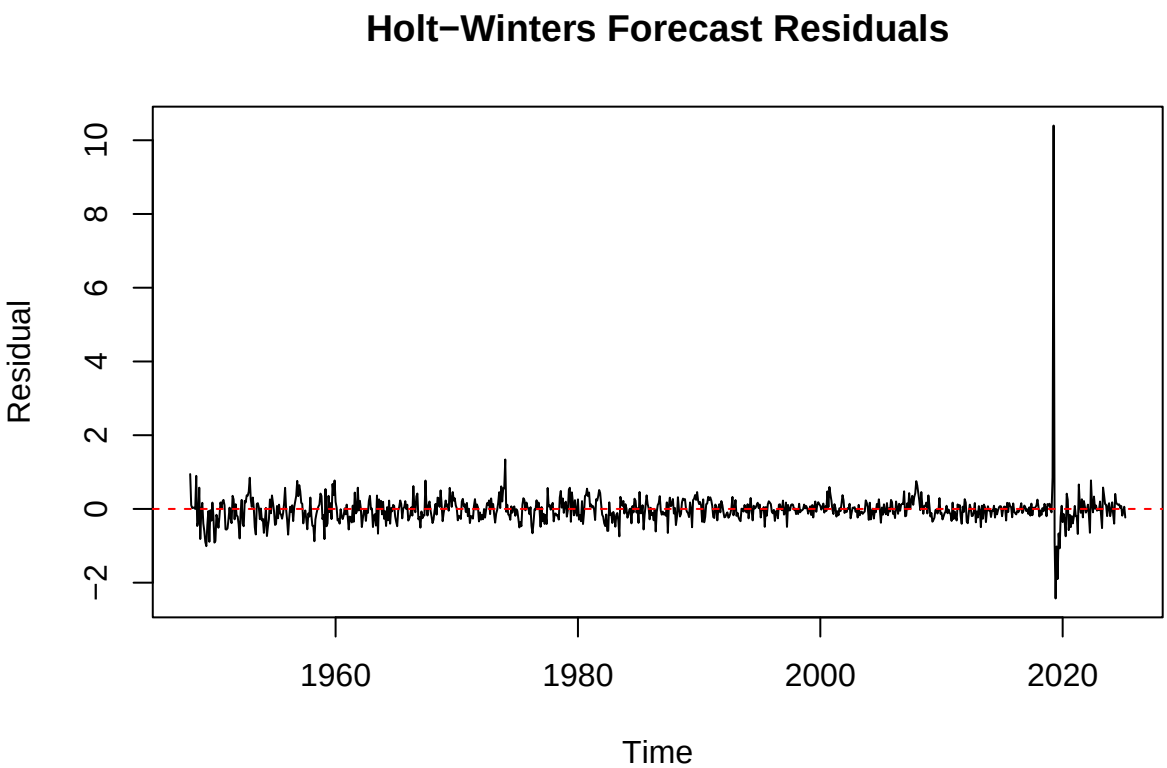
```
cat("\n=== Holt-Winters ===\n")
```

```
##
## === Holt-Winters ===
```

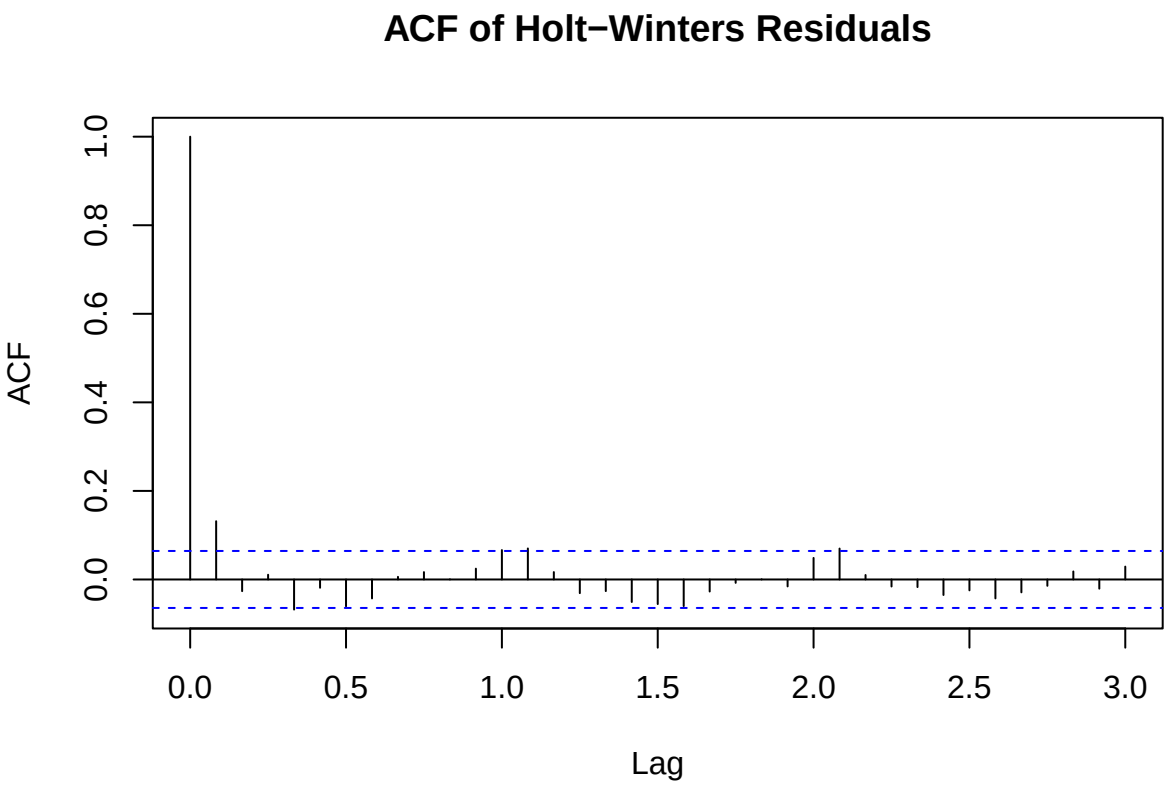
```r
print(accuracy(hw_forecast))
```

```
##                       ME      RMSE       MAE        MPE     MAPE      MASE
## Training set -0.0211553 0.4543536 0.2263107 -0.4714777 4.134302 0.2429073
##                    ACF1
## Training set 0.1315664
```

```r
# Residuals from Holt-Winters
hw_resid <- residuals(hw_forecast)
hw_resid_ts <- ts(hw_resid[!is.na(hw_resid)],
                  start = start(unrate_ts_interp),
                  frequency = 12)

# Time plot of residuals
plot(hw_resid_ts,
     main = "Holt-Winters Forecast Residuals",
     ylab = "Residual",
     xlab = "Time")
abline(h = 0, col = "red", lty = 2)
```




```r
# ACF of residuals
acf(hw_resid_ts,
    lag.max = 36,
    main    = "ACF of Holt-Winters Residuals")
```




```r
# Ljung-Box test for residual autocorrelation
Box.test(hw_resid_ts, lag = 20, type = "Ljung-Box")
```

```
##
##  Box-Ljung test
##
## data:  hw_resid_ts
## X-squared = 47.617, df = 20, p-value = 0.0004814
```

Interpretation

The Holt-Winters model captures the level, trend, and seasonal structure of `UNRATENSA` simultaneously. The smoothing parameters estimated by `HoltWinters()` indicate:

- α: how quickly the model adapts to changes in the level of unemployment.
- β: how rapidly the estimated trend slope is updated.
- γ: how strongly the seasonal pattern is re-estimated at each cycle.

Values close to zero indicate that the model relies on the full history rather than reacting sharply to recent observations, which is appropriate for a macroeconomic series with gradual structural shifts. The 24-month forecast provides a smoothing-based projection that will be compared against the ARIMA model in the final comparison section.